AI Engineering and Claude Architect · Part II, Retrieval

RAG, Lesson 3

Contextual Retrieval

Priya asks your bank's assistant how much Acme's revenue grew last quarter. The right sentence is in the index. It says "The company's revenue grew by 3%" and names neither Acme nor the quarter, so search never finds it.

Module	AI05 of 13
Lesson	3 of 7
Prerequisites	HLD M21A (embeddings and HNSW), HLD M09 (BM25), AI03 (prompt caching), AI05 L1–L2
Exam relevance	Supports only. Embedding details are out of scope for CCAR-F
Reading time	~25 minutes

Contents

1. Contextual Retrieval

Lesson 3 of 7 · RAG

Contextual Retrieval

Key idea

Chunking removes the context that makes a sentence findable. Contextual Retrieval puts it back: before indexing, a model writes a short note placing each chunk within its document. In Anthropic's tests, adding this to embeddings and BM25 and then reranking cut retrieval failures by **67%**. It costs an LLM call per chunk at ingestion, which prompt caching makes affordable.

> *Before: embedding operations (AI04 L1), hybrid search and rerank (AI04 L3), chunking (AI05 L2), prompt caching (AI03 L5). Teaches: the failure that chunking creates, Anthropic's fix, and how to run it in production. Used by: AI05 L7 (knowledge assistant design), AI08 L5 (the Java RAG service).*

The hook: the right chunk that nobody can find

Your bank's analyst assistant indexes SEC filings. A chunk from Acme's Q2 filing reads:

> "The company's revenue grew by 3% over the previous quarter."

Priya asks: "What was Acme's revenue growth in Q2 2023?"

Search method	What happens
Embeddings	The chunk means "a company's revenue grew", which is also true of every filing in the index
BM25	The chunk contains neither `Acme` nor `Q2`, so those strong terms can't match it
The model	It only sees what retrieval returned, so it answers from a competitor's filing, or says it doesn't know

> *Key idea*
>
> **This isn't a ranking bug, and a better model won't fix it.** The information linking this chunk to Acme and Q2 sits in the filing's title and headers. Chunking cut that link before anything was indexed. No search algorithm can use information that was never stored.

Intuition: a photocopied page

Someone photocopies page 212 of a report and files it in a cabinet of ten thousand pages. The page says "the company grew 3%". Without the report's cover, nobody can tell which company it describes.

You could file the whole report instead, but that is too large to read at query time. The practical fix is to write one line at the top of each photocopy: "From Acme's Q2 2023 filing, in the revenue section."

That line is all Contextual Retrieval adds.

Why the obvious fixes fall short

Fix	Why it helps	What it costs or misses
Bigger chunks	More context per chunk	Less precise matches, and more irrelevant tokens sent to the model
Overlapping chunks	Keeps sentences together	The title may still be pages away
Prepending the document title	Cheap and helps with names	Generic, and it won't name the section or period
Metadata filters	Very precise when fields exist	Needs reliable fields, and the query must supply them
Late chunking or other embedding methods	Can retain document context in vectors	Depends on the embedding model, and doesn't help BM25
Contextual Retrieval	Writes specific context into both indexes	**One LLM call per chunk at ingestion**

The title-prepending row is worth trying first. If your documents have good structure, it may be enough, and it costs nothing.

How Anthropic solved it

Anthropic published this method in its engineering post on Contextual Retrieval. Two ideas drive it:

1. **Use the model at ingestion.** A model reading the whole document can explain any chunk, and that explanation only has to be generated once.
2. **Fix both indexes.** The note is added before creating embeddings and before building the BM25 index, because each search method fails differently.

Anthropic's prompt provides the document, then the chunk, and then asks:

> "Please give a short succinct context to situate this chunk within the overall document for the purposes of improving search retrieval of the chunk. Answer only with the succinct context and nothing else."

The output is usually 50–100 tokens. The example chunk becomes:

> This chunk is from an SEC filing on ACME corp's performance in Q2 2023; the previous quarter's revenue was $314 million. The company's revenue grew by 3% over the previous quarter.

The cost problem, and why caching solves it

Each call sends the entire document, so a document with 10 chunks would be sent 10 times. **That's the same long prefix repeated, which is exactly what prompt caching is for.** The document is cached once and each chunk's call reads it from the cache. Anthropic reported a one-time cost of **$1.02 per million document tokens**, assuming 800-token chunks, 8k-token documents, 50 tokens of instructions and 100 tokens of context per chunk. It's a 2024 figure using Claude 3 Haiku, so recalculate using current prices.

What the numbers actually say

The metric is the **top-20 retrieval failure rate**: the share of relevant chunks missing from the top 20 results.

Method	Failure rate	Reduction from baseline
Baseline: standard embeddings	5.7%	–
Contextual Embeddings	3.7%	35%
Contextual Embeddings + Contextual BM25	2.9%	49%
The above + reranking (top 150 down to 20)	1.9%	67%

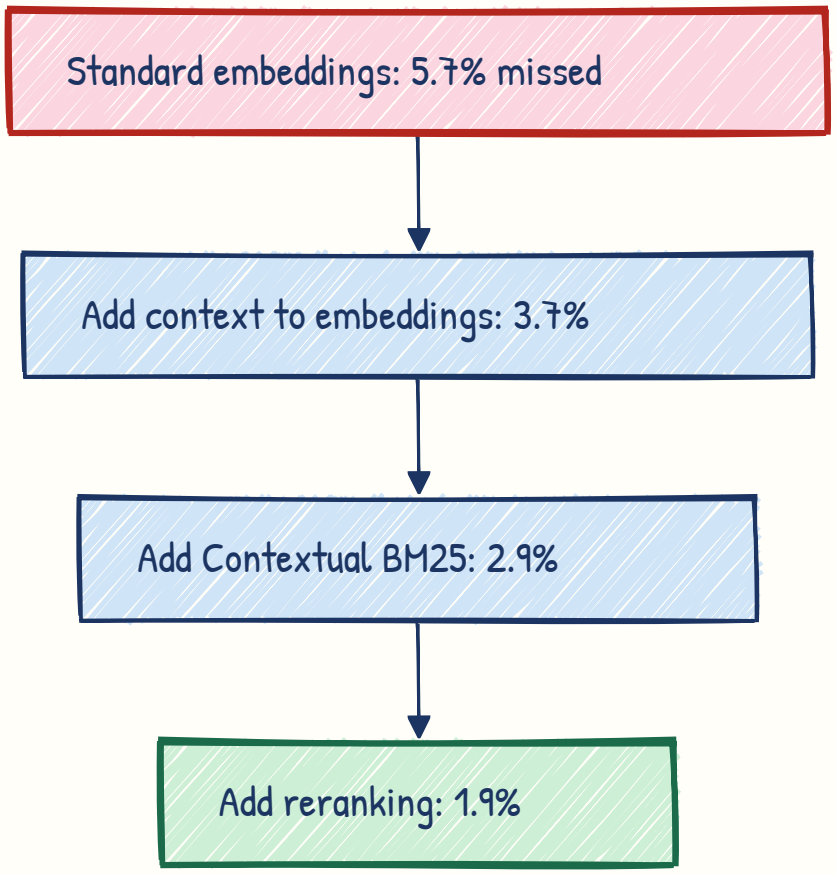


Figure 1. Each step removes failures the previous one left behind. Figures are Anthropic's reported top-20 failure rates, not a guarantee for your data.

Read this as an engineer

These results come from Anthropic's own evaluation on its chosen datasets. **Your improvement may be larger or smaller, so measure it on your own documents** (AI05 L6). The post also found embedding model choice and top-k both matter: among 5, 10 and 20, using 20 chunks performed best.

A rule that saves you the whole pipeline

As AI05 L1 discussed, Anthropic's post also says when **not** to build this. If your knowledge base is under about **200,000 tokens (about 500 pages)**, include it in the prompt and use prompt caching. **The cheapest retrieval system is often none at all.**

The production architecture

Anthropic published the method, not its internal deployment. **The design below is an engineering recommendation for running the method, not a description of any company's system.**

Start with what breaks if you do the simple thing.

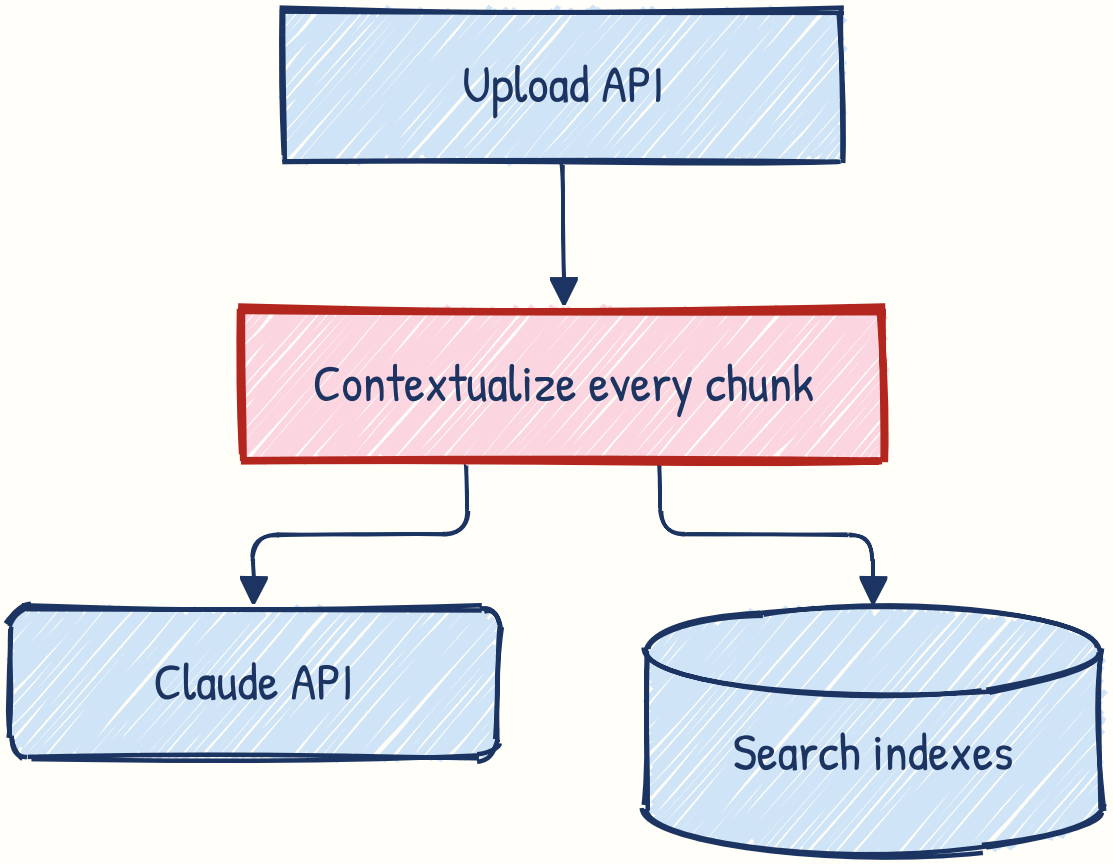


Figure 2. Baseline. Calling the LLM inside the upload request ties the user's upload to every chunk's API call.

An 800-page PDF could produce hundreds of calls. The HTTP request times out, a crash loses all progress, and an ingestion spike consumes the rate limit shared with your live assistant.

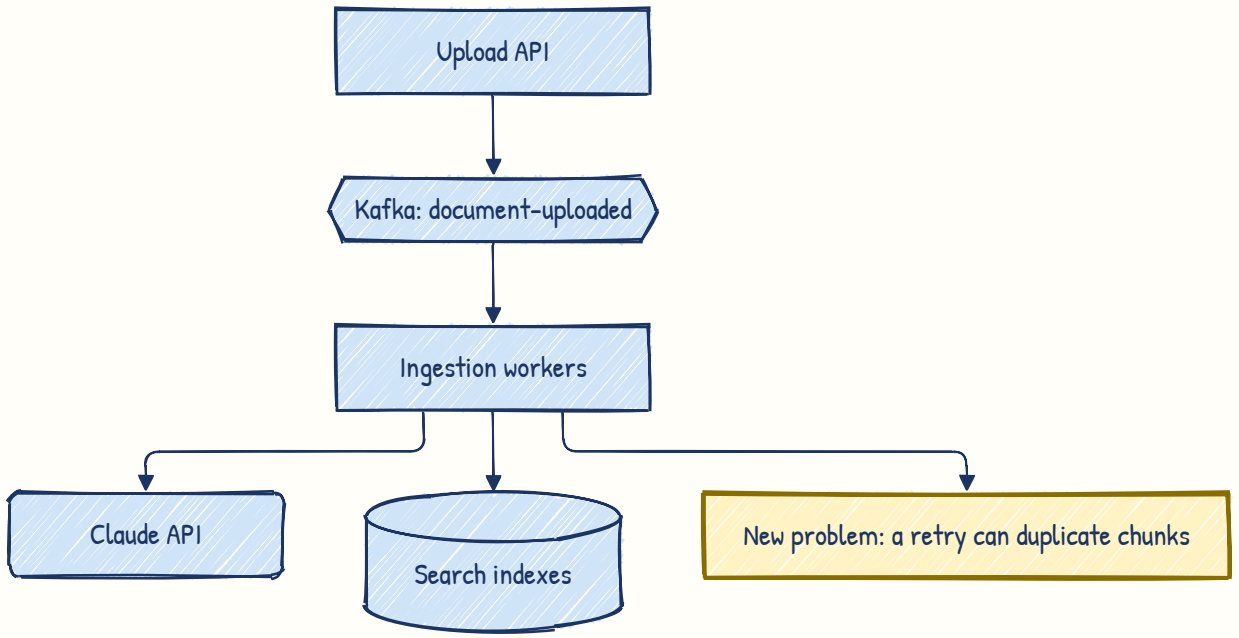


Figure 3. Improved. A queue separates upload from processing, but creates a new problem: retries and partial indexing.

The queue fixes the timeouts, using the patterns from HLD M08. A retried message can now duplicate chunks, and a half-finished document can appear in search. Both of those need explicit handling.

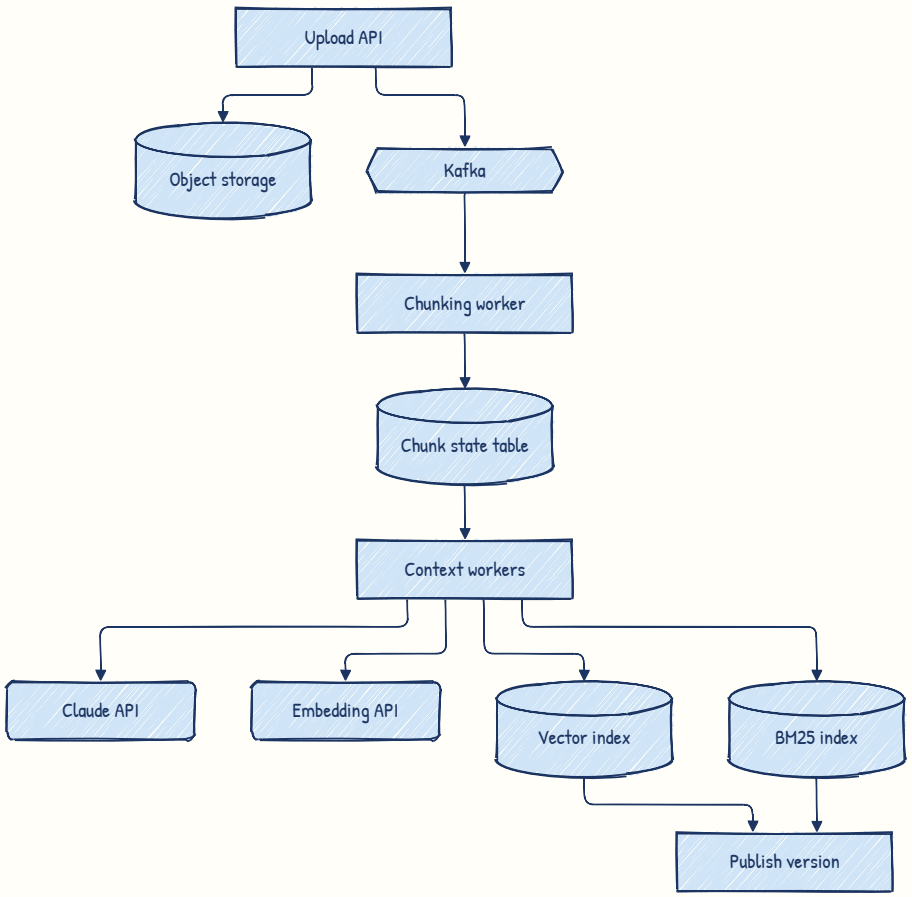


Figure 4. Production. Durable chunk state and versioned publishing make ingestion resumable and keep half-built documents out of search.

Component	Why it exists	How it talks to the others
Object storage	Keeps the original, so you can re-index without another upload	Workers read it by key
Kafka	Absorbs upload spikes and allows retries	Upload API publishes; workers consume
Chunk state table	Tracks every chunk, so a crash resumes rather than restarts	Unique key: `(doc_id, version, chunk_no)`
Context workers	Call the LLM with their own rate limit	Use a separate API key or workspace from live traffic
Vector and BM25 indexes	Each catches what the other misses	Upserts use the same key, so retries are harmless
Publish step	Search only sees completed document versions	Flips the active version after every chunk is indexed

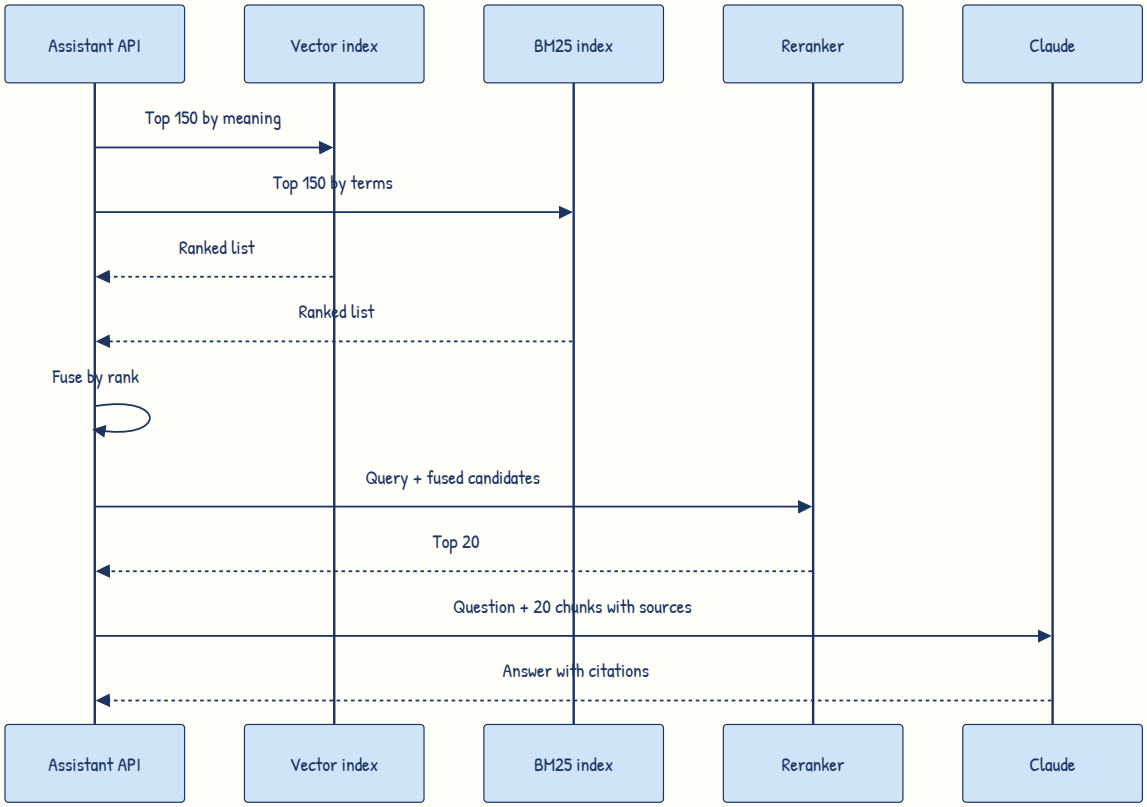


Figure 5. Query path. Both searches run in parallel, then fusion and reranking choose what the model sees.

Failures and how they're contained

Failure	Effect	Containment
Context call refused or truncated	One chunk has no note	Index the raw chunk and flag it. A missing note reduces recall, while a failed ingestion loses the document
Worker crash halfway	Partial document	State table resumes it; the old version stays live until publish
Embedding model upgrade	Old and new vectors can't be compared	Build a new index and switch over (HLD M21A)
Reranker outage	Search quality drops	Fall back to the fused order and record it, rather than failing the query
LLM invents context	A wrong note misdirects search	Treat the note as a search aid. Show users the original chunk, not the note

> *This one reaches security*
>
> Context notes are generated from document content. A malicious document can put instructions in that note, or in the chunk itself. **So retrieved text stays untrusted data at query time** (AI09 L2). Enforce document permissions in the search query, before retrieval.

Java: the two parts that matter

The context call. The document goes in a cached system block, so only the chunk varies. This compiles against `anthropic-java` 3.2.0. It hasn't been run, because that needs an API key.

```
public Optional<String> contextFor(String document, String chunk) {
    MessageCreateParams params = MessageCreateParams.builder()
            .model(model)
            .maxTokens(1024L)
            .systemOfTextBlockParams(List.of(cachedDocument(document)))
            .addUserMessage("<chunk>" + chunk + "</chunk>\n" + INSTRUCTION)
            .build();

    Message reply = client.messages().create(params);
    if (!reply.stopReason().filter(StopReason.END_TURN::equals).isPresent()) {
        return Optional.empty();
    }
    String text = reply.content().stream()
            .flatMap(block -> block.text().stream())
            .map(textBlock -> textBlock.text())
            .collect(Collectors.joining())
            .strip();
    return text.isEmpty() ? Optional.empty() : Optional.of(text);
}

private static TextBlockParam cachedDocument(String document) {
    return TextBlockParam.builder()
            .text("<document>" + document + "</document>")
            .cacheControl(CacheControlEphemeral.builder().build())
            .build();
}
```

Three lines carry a design decision:

→ `Optional.empty()` **on any stop reason except** `END_TURN`. A truncated note shouldn't be indexed as if complete, and a refusal has no usable text. The caller indexes the raw chunk.

→ `cacheControl` **on the document.** Without it, every chunk pays full price for the whole document. Check `usage.cacheReadInputTokens` is non-zero, because a prefix below the model's minimum silently doesn't cache.

→ `model` **is a constructor argument.** Anthropic used a small, cheap model. Choose yours by running an evaluation, and make it configurable.

> *Don't fan out on an empty cache*
>
> A cache entry only becomes usable after the first response starts. If 20 workers begin the same document together, all 20 can miss and pay full price. **Process the first chunk alone, then run the remaining ones in parallel.**

Combining the two searches. **Reciprocal Rank Fusion** (RRF) uses positions rather than scores, as AI04 L3 explained.

```
public static List<String> fuse(List<List<String>> rankedLists) {
    Map<String, Double> scores = new HashMap<>();
    for (List<String> ranking : rankedLists) {
        for (int i = 0; i < ranking.size(); i++) {
            int rank = i + 1;
            scores.merge(ranking.get(i), 1.0 / (K + rank), Double::sum);
        }
    }
    Comparator<Map.Entry<String, Double>> byScore =
            Map.Entry.comparingByValue(Comparator.reverseOrder());
    return scores.entrySet().stream()
            .sorted(byScore.thenComparing(Map.Entry.comparingByKey()))
            .map(Map.Entry::getKey)
            .toList();
}
```

Real output, with `K = 60` and hand-written rankings:

```
vector only = [q2-revenue, q1-revenue, competitor-revenue]
bm25 only   = [acme-note-7, q2-revenue, q1-revenue]
fused       = [q2-revenue, q1-revenue, acme-note-7, competitor-revenue]
```

Notice that `q2-revenue` wins despite topping only one list, since it ranks high in both. That is how RRF rewards agreement.

Trade-offs

Decision	You gain	It costs	Choose it when
Contextual notes	Fewer retrieval misses	LLM call per chunk; re-run when documents change	Chunks depend on their surroundings, and documents change slowly
Adding BM25	Finds exact names and codes	A second index to run	Queries contain IDs, error codes or names
Reranking	Best precision of the top results	Added latency and cost per query	Answer quality matters more than milliseconds
Skipping RAG entirely	No pipeline at all	Whole knowledge base in the prompt	Under about 200,000 tokens

When not to use Contextual Retrieval: documents that already read clearly chunk by chunk (FAQs, one-topic articles), data that changes constantly, or small collections where title-prepending already passes your evaluation.

In the real world

Anthropic's post reports the figures in this lesson, along with a linked cookbook to reproduce them. It's the vendor's own evaluation, which is why this lesson recommends running your own.

Common mistakes

1. **Showing the generated note as if it were the source.** Cite the original chunk. The note is a search aid.
2. **Adding context only to embeddings.** BM25 gains from it too; that's the step from 35% to 49%.
3. **Forgetting that a document change invalidates its notes.** Key them to the document version.
4. **Trusting published percentages.** Measure on your own data, because the gain depends on your documents.
5. **Skipping the 200,000-token check.** You might not need any of this.

What to say in an interview

Lead with the root cause: "Chunking loses context, so this is an indexing problem, not a ranking one." Then give the trade: "I'd pay one cheap LLM call per chunk at ingestion, using caching, rather than per query." Finish with how you would know: "I'd measure recall at 20 on a labeled set, before and after." That shows diagnosis, cost awareness and evaluation.

Certification lens

How this lesson maps to CCAR-F

Embedding and vector database details are **explicitly out of scope**, so don't expect questions on Contextual Retrieval itself. Three ideas here are tested in other contexts: **keeping claim-source mappings** (TS 5.6), **fix the root cause with the proportionate step** (a recurring habit in the guide's samples), and **prompt caching exists**, without implementation detail.

These are original practice questions, not real or leaked exam questions. Question 1 uses the exam's format. The others show the practice-only formats.

Q Q1 · Exam style, scenario: Multi-Agent Research System · Medium. Your document analysis subagent compresses findings into summaries. Reports read well, but reviewers can no longer tell which source supports each claim. What is the most effective fix?

A. Ask the synthesis agent to add citations from memory.

B. Require subagents to output structured claim-source mappings that downstream agents must preserve.

C. Switch to a model with a larger context window.

D. Add a final agent that searches the web for each claim.

A **B.** Attribution is lost when findings are compressed without their sources (TS 5.6). Keeping the mapping in structured output fixes the cause. A invites hallucinated citations. C doesn't change what the summaries keep. D adds cost and may attach sources that didn't support the original claim.

Q Q2 · Design decision · Hard. A team has 300 pages of policy documents, updated monthly. They plan a vector database with Contextual Retrieval. What would you ask first?

A Whether they need retrieval at all. Three hundred pages is likely below the roughly 200,000-token guideline (about 500 pages), so the whole set might fit in a cached prompt. Measure the real token count and test that first. Build the pipeline only if size, cost or access rules demand it, for example if different users may see different documents.

Q Q3 · Java debugging · Medium. `contextFor` works, but the bill shows full price for every chunk, and `cacheReadInputTokens` is always 0. Name two likely causes.

A (1) The prefix is changing between calls, for example because a timestamp or request ID was added before the cached block. (2) The document is below the model's minimum cacheable size, which silently doesn't cache. A third possibility is workers starting together before the first response creates the entry.

Revision - Contextual Retrieval

→ Chunking removes context, so some chunks can't be found by any search method.

→ Method: a model writes a 50–100 token note per chunk, added before embedding and before BM25 indexing.

→ Reported failure rates: 5.7% baseline, 3.7% with Contextual Embeddings, 2.9% adding BM25, 1.9% adding reranking (a 67% reduction).

→ Metric: share of relevant chunks missing from the top 20.

→ Prompt caching makes it affordable, because the same document repeats for each chunk.

→ Under about 200,000 tokens, skip RAG and put the content in a cached prompt.

→ Produce notes asynchronously, track chunk state, and publish only complete document versions.

→ RRF merges by rank because scores from different methods aren't comparable.

→ Measure on your own data. The published figures are Anthropic's own evaluation.

→ Embedding details are out of scope for the exam; provenance is in scope.

Your notes